Laboratório Prolog · Aula 13

Vitória Souza Santos · RA 24229424-2 · Turma ESOFT-C
Gerado em 19/10/2026, 19:51:56 · tempo registrado no cronômetro: 40:38

Preparação: Programar descrevendo, não instruindo

Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?

paradigma(L, funcional) responde L = scheme. paradigma(java, logico) responde false, pois esse fato não está na base.

Saída do ola.pl

```
Resultados esperados: linguagem(prolog), linguagem(scheme), linguagem(java);
paradigma(prolog, logico); paradigma(scheme, funcional); paradigma(java, logico) = false. As
duas consultas pedidas têm, portanto, as respostas acima.
```

0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?

paradigma(prolog, P) pergunta qual paradigma está associado a Prolog; responde P = logico. paradigma(prolog, logico) pergunta se essa associação específica é verdadeira; responde true.

0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.

Quer dizer que o programa não consegue provar isso com a base atual. Pela hipótese do mundo fechado, fatos ausentes e não dedutíveis são tratados como falsos. Isso pode ser incorreto no mundo real se a base estiver incompleta: falta de informação não prova que Java não possa ser lógico.

Etapa 1: Uma base de fatos e perguntas sobre ela

Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?

mulher(Quem) terá 7 respostas: lucia, marta, tereza, joana, bia, ana, dora, na ordem dos fatos. A conjunção terá 4 pares: (lucia, tereza), (marta, joana), (tereza, bia), (joana, ana).

Saída do familia.pl

```
Resultados principais esperados: mae(joana,jake)=true; mae(jake,joana)=false;
pai(maria,jake)=false. mae(joana,_): jake, ana. pai(_,jake): jorge. mulher(_): lucia, marta,
tereza, joana, bia, ana, dora. (mae(joana,X),homem(X)): jake. (pai(vern,Y),mulher(Y)): joana.
(mae(_,Filha),mulher(Filha)): tereza, joana, bia, ana.
```

1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.

Pergunta quem é mãe de uma pessoa que consta como mulher e quem é essa filha. São quatro respostas: Mae=lucia, Filha=tereza; Mae=marta, Filha=joana; Mae=tereza, Filha=bia; Mae=joana, Filha=ana.

1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?

A hipótese do mundo fechado: o Prolog considera falsa uma meta que não consegue provar pela base. Isso é um problema se os dados forem incompletos, pois uma relação pode ser verdadeira mesmo sem ter sido cadastrada.

1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?

Na ordem dos fatos mulher/1: lucia, marta, tereza, joana, bia, ana, dora. A ordem das cláusulas/fatos e a busca de cima para baixo do Prolog determinam a ordem das respostas.

Etapa 2: Regras: ensinando o Prolog a deduzir

Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?

tio(X,jake) responde X=paulo. primo(leo,X) responde X=jake e X=ana: Paulo, pai de Leo, é irmão de Joana, mãe de Jake e Ana.

Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?

Falham: f(X,X)=f(a,b), [A|B]=[] e 2+3=5. Tem sucesso: f(X,X)=f(a,Y), com X=a e Y=a.

Saídas de regras.pl e unificacao.pl

```
Resultados esperados de regras.pl: avo(_,jake) dá marta e vern; avo(osvaldo,_) dá joana,
paulo e bia; irmao(jake,_) falha; tio(_,jake) dá paulo; primo(leo,_) dá jake e ana;
ancestral(osvaldo,_) encontra vern, tereza, joana, paulo, jake, ana, leo e bia;
ancestral(jake,_) falha. unificacao.pl: a=a unifica; a=b não; f(X,X)=f(a,b) não;
f(X,X)=f(a,Y) unifica com X=Y=a; [H|T]=[1,2,3] unifica; [H|T]=[] não; 2+3=5 não. Resumo das
respostas esperadas pela ordem do programa.
```

2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?

A relação ancestral precisa de dois casos: base — ancestral direto, ancestral(X,Y) se progenitor(X,Y); recursivo — ancestral(X,Y) se X é progenitor de Z e Z é ancestral de Y. O caso base encerra a recursão.

2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?

Sem X \== Y, irmao(jake,_) também retorna Jake como irmão de si mesmo, porque ele compartilha consigo próprio os fatos de mãe e pai. A desigualdade exclui essa autorrelação.

2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?

Na primeira igualdade, a mesma X teria de ser a e b ao mesmo tempo, o que é impossível. Na segunda, X=a e a segunda posição impõe Y=a; portanto, unifica.

2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.

A relação Prolog pode ser consultada em direções diferentes: avo(X,jake) procura avós; avo(osvaldo,X) procura netos. Uma função comum em Java/Scheme tem direção de entrada/saída definida e não se inverte automaticamente; seria preciso implementar outro procedimento.

Etapa 3: Listas: o mesmo padrão do Scheme, em formato de relação

Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?

Quatro respostas, uma por cada divisão contígua: X=[], Y=[1,2,3]; X=[1], Y=[2,3]; X=[1,2], Y=[3]; X=[1,2,3], Y=[].

Saída do listas.pl

```
Resultados principais esperados: membro(b,[a,b,c])=true; membro(z,[a,b,c])=false; membro(_,
[a,b,c]) gera a, b, c. tamanho([a,b,c],3); tamanho([],0). concatena([1,2],[3,4],[1,2,3,4]);
concatena(_, [3], [1,2,3]) dá X=[1,2]; concatena([1],_,[1,2,3]) dá Y=[2,3]; concatena(_,_,
[1,2,3]) gera os quatro pares de prefixo/sufixo. inverte([1,2,3],[3,2,1]).
```

3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?

Caso base: tamanho([],0). Passo recursivo: tamanho([_|Cauda],N) chama tamanho(Cauda,M) e depois calcula N is M+1. Fica pendente, em cada nível, o cálculo N is M+1 até a chamada da cauda retornar M.

3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?

Há quatro posições possíveis para dividir [1,2,3] em prefixo e sufixo, incluindo listas vazias. O anexar típico do Scheme recebe as duas listas e calcula a concatenação; não usa a lista final para gerar todos os pares de partes que a formam.

3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?

Backtracking: membro tenta a cabeça na primeira regra e depois tenta recursivamente a cauda na segunda. Ao voltar pelas alternativas, gera a, b e c sem laço explícito.

Etapa 4: Aritmética: onde o Prolog deixa de ser relacional

Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?

X = 2+3 apenas unifica X com a expressão simbólica 2+3. X is 2+3 calcula e dá X=5. fatorial(N,120) dá erro de instanciação porque a regra tenta comparar N livre com zero.

Saída do aritmetica.pl

```
Resultados principais esperados: X=2+3 deixa uma estrutura simbólica; X is 2+3 dá 5; 5 =:=
2+3 é true; 2+3 = 5 falha; 7 mod 3 dá 1; 7 // 2 dá 3; 2^10 dá 1024. fatorial(5,_) dá 120;
fatorial(0,_) dá 1; fib(10,_) dá 55; soma([1,2,3,4],_) dá 10. fatorial(_,120) e _ is _+1
geram erro de instanciação.
```

4.1 Explique com exemplos a diferença entre =, is e =:=.

= unifica termos sem calcular: X=2+3 associa X à expressão. is calcula a expressão do lado direito conhecido: X is 2+3 dá X=5. =:= avalia e compara dois lados numéricos, sem vincular: 5 =:= 2+3 é verdadeiro; X =:= 5 dá erro se X estiver livre.

4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?

fatorial(N,120) não funciona com N livre porque a implementação precisa usar comparação e aritmética (N>0, subtração e is), que exigem valores conhecidos. concatena(X,[3],[1,2,3]) funciona por unificação recursiva de estruturas de lista, sem cálculo aritmético.

4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?

Ambos usam recursão e caso base. Em Haskell, fact n é uma função que devolve um valor. Em Prolog, fatorial(N,F) põe o resultado em outro argumento e descreve uma relação; porém, por usar is/comparações, esta versão funciona naturalmente com N conhecido e não é totalmente reversível.

Etapa 5: Controlando (e sofrendo com) a busca

Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?

solteiro2(X) dará false; solteiro1(X) retorna osvaldo, paulo e jake. maximo2(5,3,M) tem duas respostas: M=5 e M=3. maximo3(5,3,3) dá true no programa original, embora o resultado 3 esteja errado (o máximo é 5).

Saída de controle.pl e mensagem de ancestral_ruim

```
Resultados esperados de controle.pl: mesmo_gosto encontra (ana,caio) e (caio,ana); solteiro1:
osvaldo, paulo, jake; solteiro2: false; maximo1(5,3,M): M=5; maximo2(5,3,M): M=5 e M=3;
maximo3(5,3,M): M=5; maximo1(5,3,3): false; maximo3(5,3,3): true (erro lógico do corte).
laco_infinito.pl: ancestral_bom responde sim; ancestral_ruim não termina e estoura a pilha,
conforme aviso do próprio exercício.
```

Previsão: Quantas colorações existem com 3 cores? E com 2 cores?

Com 3 cores haverá 24 colorações. Com 2 cores haverá 0: PR, SP e MS são mutuamente adjacentes, formando um triângulo que requer três cores distintas.

Saída do mapa.pl

```
Resultados esperados de mapa.pl: 3 cores = 24 colorações; 2 cores = false (0); 4 cores =
1728. As duas primeiras com 3 cores, na ordem [RS,SC,PR,SP,MS,MG,RJ,ES], são
[vermelho,verde,vermelho,verde,azul,vermelho,azul,verde] e
[vermelho,verde,vermelho,azul,verde,vermelho,verde,azul].
```

5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?

Em tenta ana e ana, falha X \== Y porque são o mesmo termo. O Prolog volta ao ponto de escolha mais recente, amigo(Y), e tenta o próximo amigo. Ana e Bia também falham depois em gosta(Bia,cafe); o backtracking segue até encontrar ana/caio e caio/ana, que gostam de café.

5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?

solteiro1 gera um homem por homem(X) e depois verifica \+ casado(X). solteiro2 tenta a negação com X livre; como casado(vern) é provado, a negação falha antes de homem(X). Osvaldo é considerado solteiro porque não há fato casado(osvaldo); ser pai não implica casamento na base.

5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?

Na primeira cláusula, a cabeça maximo3(X,Y,X) unifica antes de executar o corpo/corte. Na consulta (5,3,3), o terceiro argumento exige X=3, mas o primeiro exige X=5; essa cláusula falha antes de chegar ao corte e a segunda aceita o caso. Corrija adiando a unificação do resultado: maximo3(X,Y,M) :- X >= Y, !, M = X. maximo3(_,Y,Y). Assim, depois do corte, o resultado incorreto 3 falha sem tentar a segunda cláusula.

5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?

ancestral_ruim faz a chamada recursiva antes de consumir um progenitor, então a busca em profundidade repete a recursão indefinidamente e estoura a pilha. ancestral_bom primeiro encontra progenitor(X,Z) e avança para Z antes de recursar, podendo alcançar o caso base.

5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?

O programa encontrou 24 colorações com 3 cores, 0 com 2 e 1728 com 4. É declarativo porque enuncia as restrições de uma coloração válida e os valores possíveis, sem programar passo a passo a busca; o backtracking encontra as soluções.

Fechamento: O que você viu e onde isso se encaixa

6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.

No funcional, anexar recebe listas e calcula uma saída. No lógico, concatena relaciona três listas: pode concatenar ou, dada a lista final, gerar suas possíveis partes. Assim, a relação pode ser usada em várias direções.

6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.

1) A ordem da busca pode impedir a terminação: ancestral_ruim estoura a pilha por recursão à esquerda, enquanto ancestral_bom termina. 2) Negação como falha é sensível à instanciação e à completude da base: solteiro2(X) falha com X livre, e solteiro1 classifica Osvaldo como solteiro porque o fato de casamento não foi registrado.

6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?

Escolheria Prolog para dedução em bases de conhecimento, sistemas especialistas e problemas de regras/restrições, como colorir o mapa. Evitaria para computação numérica de alto desempenho, interfaces ou fluxo de execução muito controlado, em que Java tende a oferecer maior previsibilidade e controle explícito.

Exercícios opcionais

Opcionais: resultado dos testes

```
Saida esperada com as solucoes acima (32 testes):
Exercicio 1
   ok      irma(ana, jake)
   ok      irma(tereza, vern)
   ok      NAO irma(jake, ana)
   ok      tia(tereza, joana)
   ok      tia(joana, leo)
   ok      NAO tia(joana, jake)
Exercicio 2
   ok      ultimo([a,b,c], c)
   ok      ultimo([7], 7)
   ok      NAO ultimo([a,b,c], b)
Exercicio 3
   ok      soma_quadrados([1,2,3], 14)
   ok      soma_quadrados([], 0)
   ok      pares([1,2,3,4,5,6], [2,4,6])
   ok      pares([1,3], [])
Exercicio 4
   ok      mdc(12, 18, 6)
   ok      mdc(17, 5, 1)
   ok      mdc(10, 0, 10)
   ok      potencia(2, 10, 1024)
   ok      potencia(5, 0, 1)
Exercicio 5
   ok      caminho(curitiba, foz)
   ok      caminho(joinville, cascavel)
   ok      NAO caminho(foz, curitiba)
   ok      NAO caminho(curitiba, paris)
Exercicio 6
   ok      3 cores: 12 coloracoes
   ok      2 cores: 2 coloracoes
   ok      1 cor: nenhuma
Exercicio 7
   ok      colega(ana, bruno)
   ok      NAO colega(ana, carla)
   ok      NAO colega(ana, ana)
   ok      ganha_mais(ana, bruno)
   ok      NAO ganha_mais(elisa, ana)
   ok      total_depto(ti, 12500)
   ok      total_depto(rh, 9000)

Desafio SEND + MORE = MONEY:
```

```
S=9, E=5, N=6, D=7, M=1, O=0, R=8, Y=2
9567 + 1085 = 10652
```

E1 Regras de família: código escrito

```
irma(X, Y) :- mulher(X), irmao(X, Y).
tia(X, Y) :- mulher(X), irmao(X, P), progenitor(P, Y).
```

E2 Listas: código escrito

```
ultimo([X], X).
ultimo([_|T], X) :- ultimo(T, X).
```

E3 As funções da Aula 12: código escrito

```
soma_quadrados([], 0).
soma_quadrados([H|T], S) :- soma_quadrados(T, S1), S is H*H + S1.
pares([], []).
pares([H|T], [H|P]) :- H mod 2 =:= 0, pares(T, P).
pares([H|T], P) :- H mod 2 =\= 0, pares(T, P).
```

E4 Aritmética: código escrito

```
mdc(A, 0, D) :- D is abs(A).
mdc(A, B, D) :- B =\= 0, R is A mod B, mdc(B, R, D).
potencia(_, 0, 1).
potencia(Base, Expoente, R) :-
    Expoente > 0,
    Expoente1 is Expoente - 1,
    potencia(Base, Expoente1, R1),
    R is Base * R1.
```

E5 Grafo com ciclos: código escrito

```
caminho(A, B) :- caminho(A, B, [A]).
caminho(A, B, _) :- liga(A, B).
caminho(A, B, Visitados) :-
    liga(A, C),
    \+ membro(C, Visitados),
    caminho(C, B, [C|Visitados]).
```

E6 Mapa menor: código escrito

```
mapa_sul(Paleta, [RS, SC, PR]) :-
    membro(RS, Paleta),
    membro(SC, Paleta),
    membro(PR, Paleta),
    RS \== SC,
    SC \== PR.
```

E7 Prolog como banco de dados: código escrito

```
% colega/2 equivale a um auto-join da tabela funcionario por departamento.
colega(X, Y) :- funcionario(X, D, _), funcionario(Y, D, _), X \== Y.
% ganha_mais/2 compara salarios em duas linhas da tabela.
ganha_mais(X, Y) :- funcionario(X, _, SX), funcionario(Y, _, SY), SX > SY.
% total_depto/2 seleciona o departamento e agrega salarios (SUM/GROUP BY).
```

```
total_depto(D, Total) :-
    findall(S, funcionario(_, D, S), Salarios),
    soma(Salarios, Total).
```